

h11



This is a little HTTP/1.1 library written from scratch in Python, heavily inspired by [hyper-h2](#).

It's a "bring-your-own-I/O" library; h11 contains no IO code whatsoever. This means you can hook h11 up to your favorite network API, and that could be anything you want:

synchronous, threaded, asynchronous, or your own implementation of [RFC 6214](#) -- h11 won't judge you. (Compare this to the current state of the art, where every time a [new network API](#) comes along then someone gets to start over reimplementing the entire HTTP protocol from scratch.) Cory Benfield made an [excellent blog post describing the benefits of this approach](#), or if you like video then here's his [PyCon 2016 talk on the same theme](#).

This also means that h11 is not immediately useful out of the box: it's a toolkit for building programs that speak HTTP, not something that could directly replace `requests` or `twisted.web` or whatever. But h11 makes it much easier to implement something like `requests` or `twisted.web`.

At a high level, working with h11 goes like this:

1. First, create an `h11.Connection` object to track the state of a single HTTP/1.1 connection.
2. When you read data off the network, pass it to `conn.receive_data(...)`; you'll get back a list of objects representing high-level HTTP "events".
3. When you want to send a high-level HTTP event, create the corresponding "event" object and pass it to `conn.send(...)`; this will give you back some bytes that you can then push out through the network.

For example, a client might instantiate and then send a `h11.Request` object, then zero or more `h11.Data` objects for the request body (e.g., if this is a POST), and then a `h11.EndOfMessage` to indicate the end of the message. Then the server would then send back a `h11.Response`, some `h11.Data`, and its own `h11.EndOfMessage`. If either side violates the protocol, you'll get a `h11.ProtocolError` exception.

h11 is suitable for implementing both servers and clients, and has a pleasantly symmetric API: the events you send as a client are exactly the ones that you receive as a server and vice-versa.

[Here's an example of a tiny HTTP client](#)

It also has [a fine manual](#).

FAQ



Whyyyyy?

I wanted to play with HTTP in [Curio](#) and [Trio](#), which at the time didn't have any HTTP libraries. So I thought, no big deal, Python has, like, a dozen different implementations of HTTP, surely I can find one that's reusable. I didn't find one, but I did find Cory's call-to-arms blog-post. So I figured, well, fine, if I have to implement HTTP from scratch, at least I can make sure no-one *else* has to ever again.

Should I use it?

Maybe. You should be aware that it's a very young project. But, it's feature complete and has an exhaustive test-suite and complete docs, so the next step is for people to try using it and see how it goes :-). If you do then please let us know -- if nothing else we'll want to talk to you before making any incompatible changes!

What are the features/limitations?

Roughly speaking, it's trying to be a robust, complete, and non-hacky implementation of the first "chapter" of the HTTP/1.1 spec: [RFC 7230: HTTP/1.1 Message Syntax and Routing](#). That is, it mostly focuses on implementing HTTP at the level of taking bytes on and off the wire, and the headers related to that, and tries to be anal about spec conformance. It doesn't know about higher-level concerns like URL routing, conditional GETs, cross-origin cookie policies, or content negotiation. But it does know how to take care of framing, cross-version differences in keep-alive handling, and the "obsolete line folding" rule, so you can focus your energies on the hard / interesting parts for your application, and it tries to support the full specification in the sense that any useful HTTP/1.1 conformant application should be able to use h11.

It's pure Python, and has no dependencies outside of the standard library.

It has a test suite with 100.0% coverage for both statements and branches.

Currently it supports Python 3 (testing on 3.8-3.12) and PyPy 3. The last Python 2-compatible version was h11 0.11.x. (Originally it had a Cython wrapper for [http-parser](#) and a beautiful nested state machine implemented with `yield from` to postprocess the output. But I had to take these out -- the new *parser* needs fewer lines-of-code than the old *parser wrapper*, is written in pure Python, uses no exotic language syntax, and has more features.

It's sad, really; that old state machine was really slick. I just need a few sentences here to mourn that.)

I don't know how fast it is. I haven't benchmarked or profiled it yet, so it's probably got a few pointless hot spots, and I've been trying to err on the side of simplicity and robustness instead of micro-optimization. But at the architectural level I tried hard to avoid fundamentally bad decisions, e.g., I believe that all the parsing algorithms remain linear-time even in the face of pathological input like slowloris, and there are no byte-by-byte loops. (I also believe that it maintains bounded memory usage in the face of arbitrary/pathological input.)

The whole library is ~800 lines-of-code. You can read and understand the whole thing in less than an hour. Most of the energy invested in this so far has been spent on trying to keep things simple by minimizing special-cases and ad hoc state manipulation; even though it is now quite small and simple, I'm still annoyed that I haven't figured out how to make it even smaller and simpler. (Unfortunately, HTTP does not lend itself to simplicity.)

The API is ~feature complete and I don't expect the general outlines to change much, but you can't judge an API's ergonomics until you actually document and use it, so I'd expect some changes in the details.

How do I try it?

```
$ pip install h11
$ git clone git@github.com:python-hyper/h11
$ cd h11/examples
$ python basic-client.py
```

and go from there.

License?

MIT

Code of conduct?

Contributors are requested to follow our [code of conduct](#) in all project spaces.